AI AND SOFTWARE GUILD

Software Development Vertical

RECRUITMENT APPLICATION 2026

2 Let Us Know You . . .

2.1. Technical Background

Describe your journey with programming and software development so far. Include the languages, tools and frameworks you are familiar with, your prior experiences; and what motivated you to start.

Ever Since I was a Kid, I've had this Weirdly Specific Obsession with computers and code, and coincidentally my mom was a computer science professor.

(Peak Nepo Baby Origins??)

Then she went full "mentor mode" and started teaching me how to code and, more importantly, how to think about what the computer is actually doing behind the scenes.

My first languages were HTML, CSS and JavaScript, and somewhere down the line I picked up Python, a bunch of Weird Math I didn't fully understand at first *(and still did not, my mom didn't know that you can't teach discrete Math to a 9-year old)*, and libraries like Pandas, Numpy, Sympy and ~8 others.

Tools? - I mean, at this point I basically know how to (ab)use LLMs, VS Code, Git/GitHub, Vercel, Notion, etc. to get things shipped slightly faster than my impostor syndrome.

My first "serious" project was a Solar System info website for a school CS group project where I, very humbly, ended up doing like 90% of the work [I(Hamilton) wrote the other 51 ahh].

Since last year, I've been vibe-coding a handful of websites and throwing some of

them onto LinkedIn (pls connect with me on linkedin) whenever I remember that "building in public" is a thing.

My motive to get into the Software Guild is basically to level up from this casual WebDev phase I'm in right now into something more structured, competitive and high-quality and if I can learn a lot, ship cool stuff then I could *potentially win* a few competitions along the way.

2.2. Interest & Motivation

Why are you applying to the Software Development vertical of the AI and Software Guild?

Why am I applying to the Software Development vertical? (I mean they are hiring ?) Because apparently "vibe-coding random side projects at 2 AM" is not a long-term career plan.

and I'd like to upgrade from Solo Developer Arc to "actually works with a team and ships Real Things" Arc(i need to touch some grass).

Right now I write a lot of code on my own, break stuff, fix stuff, repeat, but I'm very aware that I'm missing out on proper reviews, structure, and people who will *politely bully me* into better practices.

From what I've heard, the Software Guild is basically the place where all the "I like building cool things and I am autistic" people go to suffer together in a productive way.

I want to be in rooms where people talk about architecture, testing, design decisions and not just "bro, how to center a div".

I'm also unironically excited to have my code roasted, learn from seniors who've actually shipped things, and understand what "good codebase" means beyond my GitHub repos.

2.3. Hackathon Experience

List the hackathons you have participated in along with results. Describe your approach while participating in a hackathon.
(Note: It is fine if you haven't participated in any hackathon — just mention the ones you are planning to participate in this year.)

Once upon a time, when I was at the tender age of 18 (so like, 2024 lore), I tried my first ever hackathon at IITH.

I got grouped into a team with 3 very serious, very sweaty final-year dudes — 2 CS and 1 Chemical (because of course there is always a chemical guy in the party composition).

There were 4 problem statements that I have absolutely wiped from my memory, and those 3 seniors basically speedran the whole project while I tried to keep up and not look completely lost because I only knew HTML at that point.

We somehow built something that almost worked… only to realise we had heroically missed the submission deadline(FAHHHH…).

So no, we didn't win anything there unless you count "emotional damage + experience" as a prize, but that hackathon hard-unlocked something in my brain.

I realised I weirdly enjoy the combo of sleep deprivation, bad coffee and shipping half-broken features at 3 AM while someone yells "I SWEAR, IT WORKED FIVE MINUTES AGO".

Now my approach to hackathons is basically:

1. speedrun understanding the problem
2. brutally cut scope
3. get the most boring working version out first before my brain

starts daydreaming about microservices and animations.

I shamelessly (ab)use boilerplates, LLMs and templates to skip the "stare at empty repo" phase and jump straight into building.

Then, once we have something that runs without exploding, I start thinking about polish, UI, edge cases and "how do we not embarrass ourselves in the demo".

In short:

1. minimal overthinking
2. maximum shipping
3. absolutely no heroic last-minute "trust me it'll work in prod" experiments

. . . Hopefully.

2.4. AI Use-case & Opinion

What is your opinion about using AI tools for software development? How do you utilise AI tools? What are the pros and cons according to you?

Personally, I think AI tools for dev are like giving every programmer a slightly unhinged senior who answers instantly and wrong confidently, gaslights you occasionally, but still somehow makes you 2x faster if you don't blindly trust it.

They're not a brain replacement, more like a brain amplifier with built-in debuffs such as Hallucination.

I use LLMs for all the unglamorous-but-important stuff:

- generating boilerplate
- Altering the UI/UX(i have bad taste in colour)
- writing boring CRUD
- setting up configs
- refactoring chunks of code
- drafting tests

AI AND SOFTWARE GUILD

- explaining weird error messages like I'm five

Half the time I'm treating it as a debugger — I paste my code, ask "why does this hate me", and then steal the 20% of its answer that actually makes sense.

When I'm exploring a new stack, I'll also use it to spin up tiny example snippets so I can focus more on "how does this fit into my project" instead of "how do I remember this syntax".

PROS

- speedruns everything boring
- lets you prototype ideas stupidly fast
- helps you read unfamiliar codebases
- gives you instant feedback when you're stuck at cursed hours of the night
- It also forces you to be precise with your prompts-> which secretly trains you to think more clearly about requirements and architecture.
- Gives me therapy, when i need(actually i need therapy)

CONS

- it hallucinate-code s with full confidence
- sometimes gives subtly wrong solutions that only explode in edge cases
- if you just copy-paste without thinking, you become a "professional prompt typist" instead of someone who actually understands what's running in prod.
- There's also the fun little problem of privacy and leaking sensitive code if you're careless, so I treat anything serious like "do not feed this to the robot without thinking twice".(i clearly did not share my API keys with LLM's, no way)

So my philosophy is: (ab)use AI for speed, boilerplate and exploration, but be absolutely ruthless about checking the output — tests, reading line by line, and making sure that at the end of the day, my brain (sadly) is still the final code reviewer and not the model.

3 Software Development Fundamentals

Instructions: Please answer all four theoretical questions below with clarity and depth.

3.1. Web Architecture

What is client-server architecture? Provide a full overview of what happens when you enter a URL in your browser. How do you manage a heavy traffic on a website? What are the types of 2 types of scaling designs which are used?

Client-server architecture is basically "browser is the spoiled kid, server is the overworked parent".

The client (A.K.A - your browser/app) keeps asking for stuff ("give me this page", "give me this data"), and the server does the heavy lifting — runs logic, talks to the database, and sends back responses so the client can render pretty pixels.

Now, when you smash 'Enter' on a URL in your browser, a whole drama happens behind the scenes:

The browser first checks: "Do I already have this cached somewhere?" If yes, free speed.

If not, it goes around asking the DNS system like a confused fresher: "Excuse me,Umm…, who is *example.com*?" until someone gives it the actual IP address.

Once it knows where to go, it knocks on the server's door with TCP, and for HTTPS they also do the secret handshake (TLS) so no one eavesdrops on the gossip.

The browser then sends an HTTP request — basically a very detailed "Hey, can you get me this thing?" with headers, cookies, maybe a body. A load balancer might be standing at the gate like a bouncer, deciding which backend server has the energy to deal with you

The chosen server reads the request, panics a little, then talks to databases and other services, runs all the logic, and prepares an HTTP response (HTML, JSON, etc.).

That response goes back to your browser, which then parses the HTML, builds the DOM, fetches CSS/JS/images/fonts via more requests, and runs your JS to finally show you the page.

From your POV: "page loaded". From the web's POV: "20 network calls later…"

When traffic gets heavy and your main server starts crying in a corner, you bring in reinforcements so it doesn't die:

CDNs are like regional food courts — they store and serve static stuff (images, CSS, JS) from places closer to the user so everyone doesn't rush the main kitchen.

Caches are the short-term memory of the system: browser cache, CDN cache, reverse proxy cache and in-app cache (like Redis) so the server doesn't recompute the same thing 10,000 times.

Load balancers are traffic police at a busy junction, directing incoming cars (requests) to different lanes (servers) so one road doesn't get destroyed.

Databases get help via indexes, read replicas and queues so heavy work (emails, reports, ML jobs) happens calmly in the background instead of right in the middle of a user request.

To scale this circus, you've got two main options:

Vertical scaling (scale up)	Horizontal scaling (scale out)
make one server absolutely jacked: more CPU, more RAM, more disk. Mentally simple ("just buy a bigger machine"), but there's a hard upper limit and it gets expensive.	instead of one ultra-buff server, you hire many normal servers and teach them to work together using load balancers and coordination. Harder to manage, but this is how you survive when suddenly 1 lakh users decide to click at the same time.

3.2. API Protocols

What are APIs? Why are they important? What are RESTful APIs? Define the major CRUD HTTP methods.

APIs are like *professional translators* standing between two awkward services that don't speak the same language but really need to work together.

Instead of each system dumping its internal mess on the other, the API says, "Ask me in this format, I'll clean it up and give you something usable back."

That's why they're important: they hide internal chaos, expose only what's needed, and let frontends, backends and third-party services collaborate without knowing each other's internals.

A RESTful API is not an API that sleeps, in fact it is an organized translator who insists on using proper grammar and a fixed script.

They treat everything as resources with clear addresses like /users/67 and use standard HTTP methods to express intent:

GET - the librarian: "I'll fetch what you asked, nothing gets changed."

POST - the receptionist: “I’ll register this new thing in the system.”

PUT - the replacer (ik it’s not a job): “Throw out the old record, use this updated one.”

PATCH - the detail fixer (i ran out of jobs) : “I’ll just edit the specific bits you mentioned.”

DELETE - the cleaner: “This entry is gone. You never saw it.”

3.3. Database Management

What are the two types of databases, with examples? Describe the use case of each.
What is the structure of a SQL database?
Research how a SQL database (like PostgreSQL or MySQL) uses B-Trees for indexing. We know that adding an index makes SELECT queries faster.
However, if you are performing 10,000 INSERT operations per second, having too many indexes will crash your performance.
Based on your research into B-Tree node splitting, explain what is physically happening to the database file on the disk that makes writes slower when an index is present.

There are two main species of databases: relational (SQL) and non-relational (NoSQL).

SQL ones are the organised toppers - think PostgreSQL, MySQL - with tables, rows, columns, schemas and strict rules	NoSQL ones like MongoDB, Redis are the more “vibes-based” cousins that store documents, key-values, graphs or wide columns and trade some structure for flexibility and scale.
I’d use SQL when I care about relationships, consistency and structured data	NoSQL when I’m dealing with huge, flexible or unstructured data (logs, events,

(users, orders, transactions)	caches, JSON blobs) where schema can't sit still.

A SQL database is structured as: database → tables → rows → columns with constraints, indexes and relationships (primary keys, foreign keys) tying everything together.

Under the hood, engines like PostgreSQL/MySQL store table data and indexes on disk in pages/blocks, and indexes (for many types of columns) are implemented using B-Trees.

In a B-Tree index, values are stored in sorted order in nodes, each node pointing to children, so SELECT can walk the tree instead of scanning the whole table.

This is why SELECT gets faster with an index: you go from "read almost everything" to "walk a short tree path and jump directly to relevant rows".

But for heavy INSERTs (like 10,000 per second), the index becomes that one picky librarian who wants everything sorted all the time.

Every time you INSERT, the database not only writes the new row to the table but also has to insert the indexed value into the correct place in the B-Tree.

Physically, that means: reading index pages from disk, modifying them, and sometimes splitting nodes when a page is full, creating a new page, moving half the keys, updating parent pointers, and writing multiple pages back to disk.

With many indexes, each INSERT triggers multiple B-Tree updates and splits, so instead of one simple append-style write, your disk is doing extra random I/O and page rewrites, which tanks write performance.

3.4. GIT Fundamentals

Research the difference between git merge and git rebase.
Now, consider this:
If you rebase a branch, the commit messages might look the same, but the Commit Hashes change.
Explain why this happens by referencing how Git stores objects in its internal database.
If a commit is just a snapshot of a tree, why does changing its parent change the identity of the commit itself?

Git merge vs Git rebase is basically "let's blend timelines" vs "let's rewrite history and pretend we were always this organised".

A merge takes two branches and creates a new merge commit with two parents, keeping the original commit graph as it really happened, just tying them together.

A rebase takes your branch, lifts your commits off their old parent, and replays them on top of a new parent, as if you had started from there all along.

Inside Git's little brain, every commit is an object identified by a hash (SHA-1/SHA-256) of its contents plus metadata.

That commit object stores: a pointer to a tree (snapshot of the files at that commit), the parent commit hash(es), author, message, timestamps, etc.

The hash is calculated from all of that together, not just the tree, but also the parent hash and metadata.

So when you rebase, you are not "moving" the same commits; Git is creating new commits with the same messages and similar changes but different parents.

Since the parent hash is part of the data that gets hashed, changing the parent changes the resulting hash, so Git

sees them as completely new objects with new identities.

That's why after a rebase, the commit messages may look identical, but their hashes all change: the history (who their parent is) changed, so the commit's identity changed too.